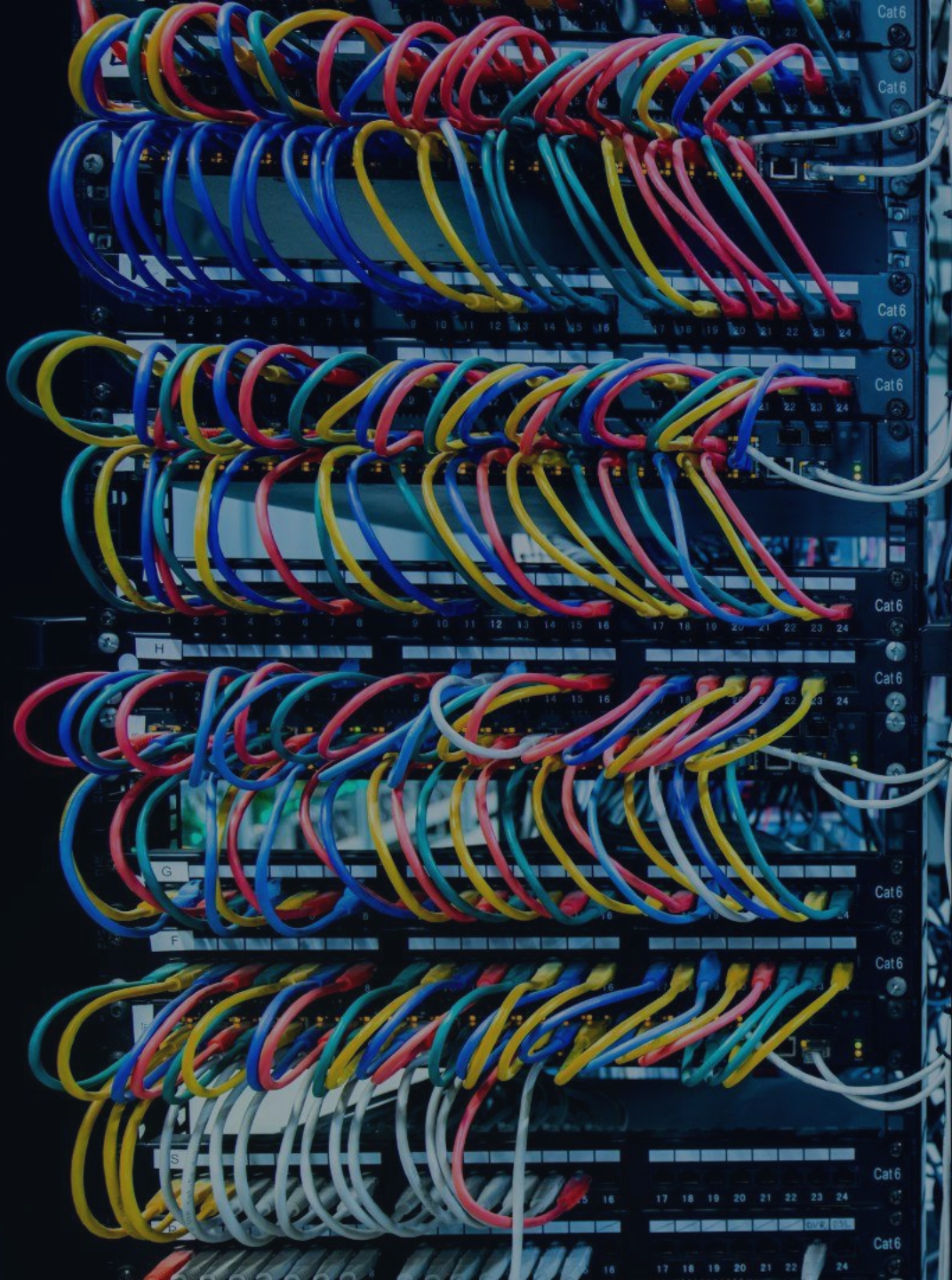




Context Model Protocol (MCP)

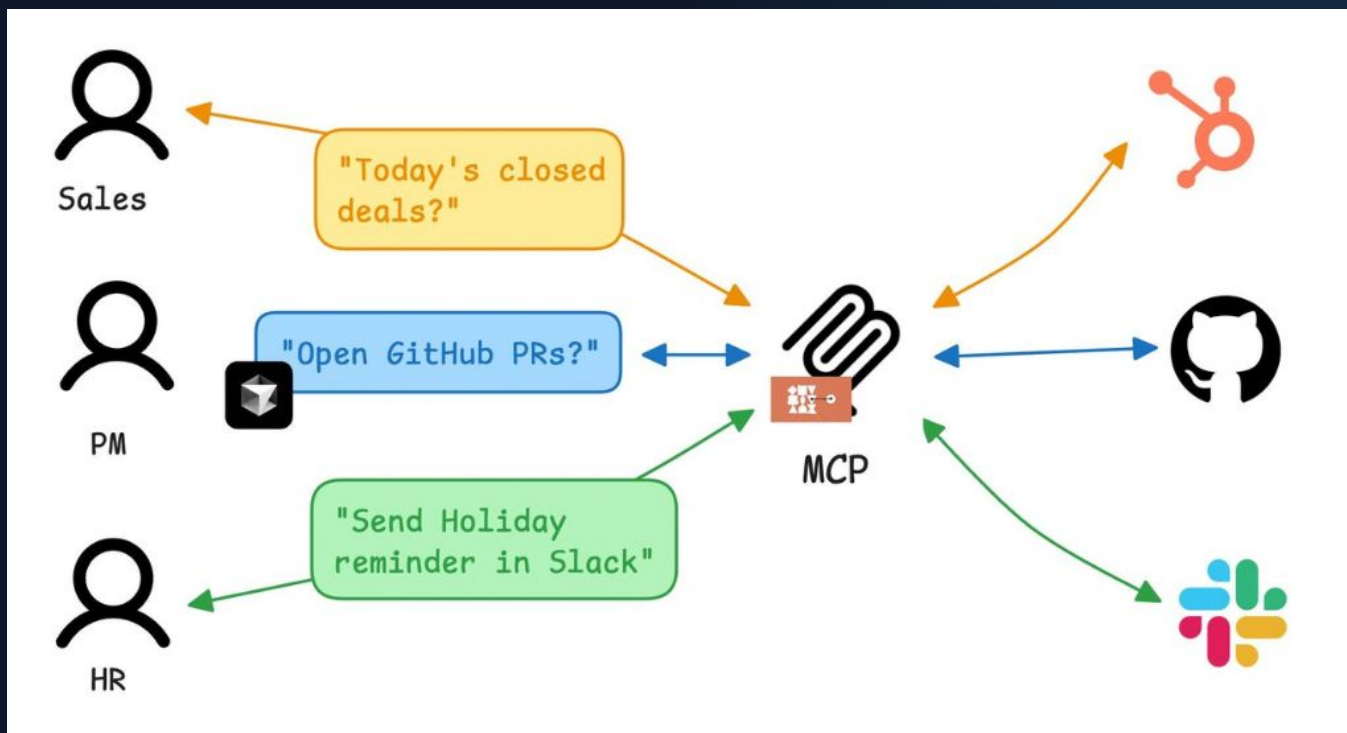


¿Qué es MCP?

→ Protocolo universal estándar

MCP (Model Context Protocol) es un estándar de código abierto para conectar aplicaciones de IA a sistemas externos.

Mediante MCP, aplicaciones de IA como Claude o ChatGPT pueden conectarse a fuentes de datos, herramientas y flujos de trabajo lo que les permite acceder a información clave y realizar tareas.



MCP Server

Los MCP servers son programas que exponen capacidades específicas a las aplicaciones de IA.

Algunos ejemplos comunes incluyen servidores de sistemas de archivos para el acceso a documentos, servidores de bases de datos para consultas de datos, servidores GitHub para la gestión de código, servidores Slack para la comunicación en equipo y servidores de calendario para la programación de tareas.



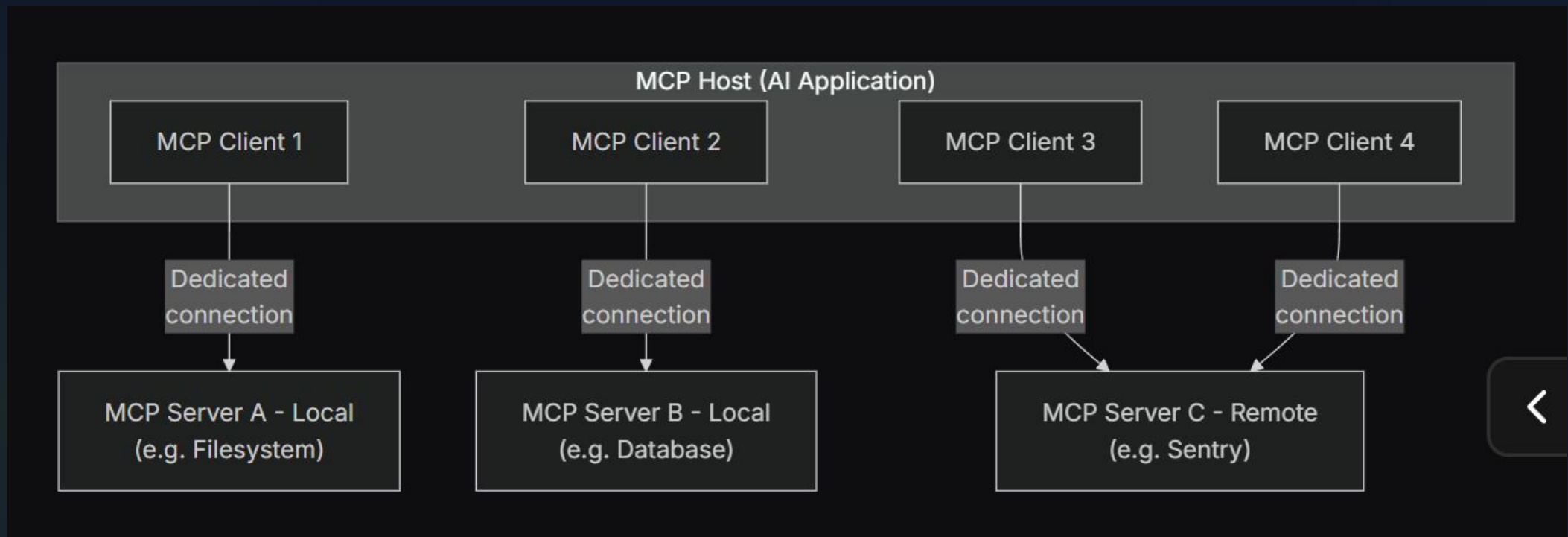


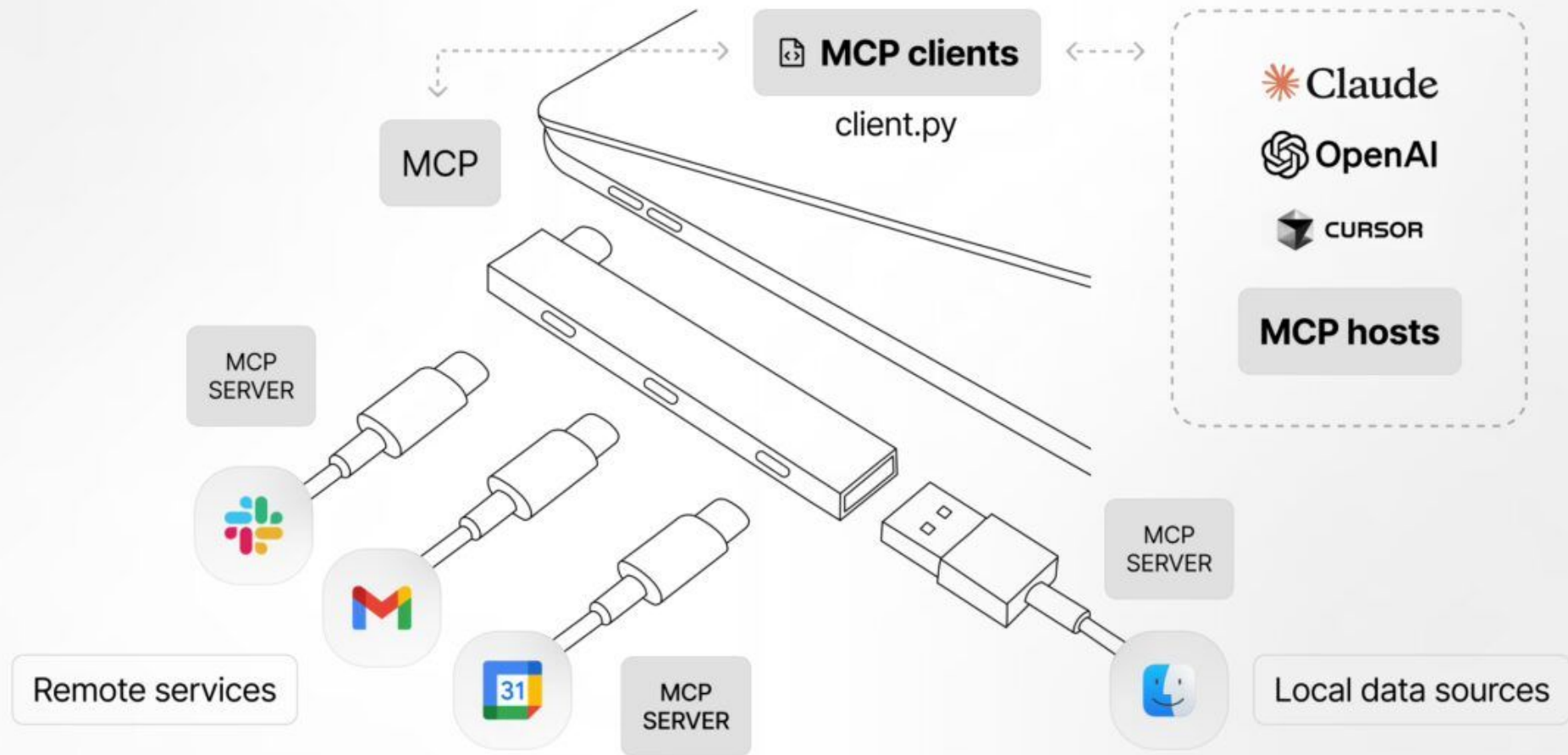
Arquitectura del Sistema

MCP Host - La aplicación de IA que coordina y gestiona uno o varios clientes de MCP.

MCP Server - Un programa que proporciona contexto a los clientes de MCP.

MCP Client - Un componente que mantiene una conexión con un servidor MCP y obtiene contexto de un servidor MCP para que el host MCP lo utilice.





Capa de Transporte

Utiliza dos métodos principales para la comunicación cliente-servidor:



StdIO (Entrada/Salida)

Optimizado para recursos locales. Ofrece una transmisión de mensajes rápida y síncrona, ideal para integraciones directas en el sistema.



Transporte HTTP en streaming

Utiliza el protocolo HTTP POST para el envío de mensajes del cliente al servidor, con la opción de usar Server-Sent Events para la transmisión de datos en tiempo real.

Permite la comunicación con servidores remotos y admite métodos de autenticación HTTP estándar, como tokens de portador, claves API y encabezados personalizados.

Capa de Transporte

Cuando una plataforma debe gestionar múltiples workers y clientes, la capa de transporte debe ampliarse con:

Supervisión de Procesos

Creación, monitorización y reinicio automático de procesos de trabajo.

Afinidad de Sesión

Garantiza que mensajes de una sesión lleguen al mismo worker para mantener el estado en memoria.

Solicitud-Respuesta

Enrutamiento preciso de respuestas desde los workers al cliente de origen.

Contrapresión (Backpressure)

Evita el agotamiento de memoria sincronizando el ritmo entre productores rápidos y consumidores lentos.

Entrada Multimodal

Conectividad flexible: StdIO (incrustación), Sockets Unix (IPC local) y TCP (red).

Capa de Datos

Todos los mensajes son objetos JSON-RPC 2.0 encapsulados como NDJSON a través de un flujo de bytes bidireccional.

NDJSON

JSON delimitado por saltos de línea (NDJSON) es un formato para almacenar datos estructurados que se pueden procesar registro por registro. Cada línea es un valor JSON válido, separado por saltos de línea (LF -> \n).

```
{"name": "Alice", "age": 30}  
{"name": "Bob", "age": 25}  
{"name": "Charlie", "age": 35}
```

** Nótese la falta de llaves [] para encapsular el mensaje*

JSON-RPC

Es un protocolo de llamada a procedimiento remoto (RPC) ligero y stateless. Es independiente del protocolo de transporte, ya que sus conceptos pueden utilizarse dentro del mismo proceso o diversos entornos.

```
--> REQ {"jsonrpc": "2.0", "method":  
"subtract", "params": {"minuend": 42,  
"subtrahend": 23}, "id": 3}
```

```
<-- RES {"jsonrpc": "2.0", "result": 19, "id":  
3}
```

```
<-- RES {"jsonrpc": "2.0", "error": {"code":  
-32700, "message": "Parse error"}, "id": null}
```


Inicialización

Request

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "initialize",
  "params": {
    "protocolVersion": "2025-06-18",
    "capabilities": {
      "elicitation": {}
    },
    "clientInfo": {
      "name": "example-client",
      "version": "1.0.0"
    }
  }
}
```

Response

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": {
    "protocolVersion": "2025-06-18",
    "capabilities": {
      "tools": {
        "listChanged": true
      },
      "resources": {}
    },
    "serverInfo": {
      "name": "example-server",
      "version": "1.0.0"
    }
  }
}
```

Uso de herramientas

Request

```
{
  "jsonrpc": "2.0",
  "id": 3,
  "method": "tools/call",
  "params": {
    "name": "weather_current",
    "arguments": {
      "location": "San Francisco",
      "units": "imperial"
    }
  }
}
```

Response

```
{
  "jsonrpc": "2.0",
  "id": 3,
  "result": {
    "content": [
      {
        "type": "text",
        "text": "Current weather in San Francisco: 68°F, partly cloudy with light winds from the west at 8 mph. Humidity: 65%"
      }
    ]
  }
}
```

Fuentes de información

- <https://modelcontextprotocol.io/docs/getting-started/intro>
- <https://modelcontextprotocol.io/docs/learn/architecture>
- <https://cloud.google.com/discover/what-is-model-context-protocol>
- <https://arxiv.org/pdf/2506.13538>
- <https://www.techrxiv.org/doi/pdf/10.36227/techrxiv.177273715.53914468/v1>
- <https://ndjson.com/>
- <https://www.jsonrpc.org/specification>